

Grove ingest

The Grove Ingest project is a typescript based system for ingesting data from various other sources via the composio api.

What is Grove?

Grove is an Urbit app that stores Markdown (Md) files where you can then view and edit said files. You can also link your ai agents to grove, giving them reading and writing permissions to these files. When I was first testing out Grove, I noticed that it was in a generally usable state. I found several bugs, but these were generally small things. When I was building against it, I found its limitations, and then how to work around these to benefit my project.

Project structure

The `examples/` folder contains various example output files. A calendar event, a github PR, a github issue, and a starbucks promotional email.

The `output/` folder is the default output folder when the local option is chosen.

- The `src/` folder contains everything else
 - The `src/cli/` folder contains the internals for the command line version to run the app.
 - The `src/web/` folder has all the files needed to run the web version. This includes a tiny local running server for a custom folder reader (see later).
 - The `src/config/` folder has the functions for the config. The format of the config, setting the config, and keyring methods.
 - The `src/connectors/` folder is the primary sector of this project. This houses the systems for each type to pull data through composio.
 - The `src/renderer/` folder takes the pulled data and converts into the normalized format found at `src/renderer/markdown.ts`
 - The `src/sinks/` folder finally takes that normalized format and is used either locally or sent to grove.

The Command Line Interface (CLI)

The command line version of this app utilizes Prompts from Clack, which does low-level primitives for command line apps. The command line sends you down a path to select the exact setup the user wants. The pros of this is that it's easy to use and setup, without needing anything extra besides starting it. The cons is that nothing is saved and you need to go through the prompts every time.

The Web version

The web version of this app utilizes TypeScript React to display and make a website for typescript. The website version is, to be blunt, majority generated by Claude. I am not the biggest fan of web development in typescript, so I first worked it out in Swift, then had Claude translate it into typescript. The theming of the website was Solarized, however the core components can be in any theming.

The custom folder reader (web mode only)

On non chromium browsers (ex. Safari, firefox), these browsers only give websites read permissions. For locally writing, this is bad as the files cant be written. This is why it hosts a tiny server that the website can talk with to get the folders it can write to.

The connectors

In the connectors folder, there are sub folders for each of the ingestion types. And in these folders, there is at least 1 file to do composio connections. Github is currently the only one that has the ability to do a local connection that doesn't use composio. These first check if the config is valid. Once it's valid, it then checks to see if the connector is active. Being active means that composio is linked correctly to the connection. For example, the github connection only becomes active once composio has been linked to your github.

The renderer

This follows a similar file structure as connectors. It takes both the config and the data pulled from the connector and converts it into a normalized format that could be used in the sinks. This normalized format has the following required bits of data.

- Provider (ex. gmail)
- External Id – a unique id provided by the provider.
- Title
- Body – though technically this isn't needed, as seen if something doesn't have a body where it is defaulted to *"No body provided"*
- Created At (ex. 2000-01-01T00:00:00Z)
- Updated At (ex. 2000-01-01T00:00:01Z)
- Synced At (ex. 2026-01-01T17:55:55Z)
- Kind (ex. email)

There are then the optional fields for each project type, allowing other data to be ingested. For instance, the Google Calendar ingest has a start date and an end date, while github has the issue number and repo name.

The sinks

There are two sinks, a local sink and a Grove sink. The local sink writes markdown files locally, writing to the location the user chose. The Grove sink writes the files to the Grove location using its api.

The file names

Let's take the file name `Re: Cool thing -- a1b2d2f8.md`. This first takes the title and sanitizes it. Various operating systems do not allow certain symbols in the file name. As such, this converts these symbols to unicode equivalents. The second part, the set of characters, is a string to give each file a unique id, to prevent the chance of overwriting. This is generated by taking the provider, kind, and external id in the format of `provider:kind:externalId`. This is then converted to sha256, then converts it to hex. After this, it's stripped down to 8 characters.

Auth

This project uses composio OAuth to do everything composio with the exception of the github native, using the Octokit OAuth. The Composio key and github auth value are both stored in the Systems keyring, to keep everything safe and secure. If, for whatever reason, that key is no longer valid (manually discontinued, or stale), then the system will remove it from the keyring, and allow you to input the new one. There is also a manual option to replace it.

The actual syncing

Syncing is done relatively simply. On the CLI, you'd need to go through the whole path to sync again, but the Web version only needs the button to be clicked again due to how websites work. When it's synced locally, it first checks if the updated at dates line up. If they *don't* then it syncs, else it just ignores the file. And if the file doesn't exist, then it just makes a new file.

Failure protection

If the config is malformed or incorrect, the project will crash. The config is required and if it's incorrect, the entire project won't work. If composio auth fails, this will be errored correctly, pasting the reason why, and stopping safely. If, say one file of 20 in an ingest is incorrect, all the files will be synced up until that malformed file, where then it will throw an error.

Limits

Using the native api, there are generally zero limits, or restrictions, besides rate limiting. However, with Composio, the free plan is limited to 20K calls per month. Whilst this will most likely rarely ever get hit, it is still a valid concern to have at some point. After this, it is 29 USD/mo with one tool call (what is used in this) being 0.0003 per call. This gives you an extra 200k calls.

Composio viability

Compared to the manual options, Composio is the obvious choice. Native github integration is about double the length compared to Composio. Composio's toolkit is sometimes limited, and having to work around that can get a little annoying, but it's usually small things. The 20k limit of Composio is, realistically, very hard to hit, especially with how Grove Ingest is built. If I had to do this project again, I would choose to use Composio, or maybe a similar project, but composio does seem to be the best out there.

Composio has some pros and some cons. One of the pros is how well documented it is. Being able to easily access all the documentation and then use it makes everything so much easier. Its functions being easy to use and remember helps a lot too. However, Composio also has some tedious cons that directly using the api wouldn't have. One of these is syntax. Currently, you have to have a previous run up to know the format of the output, which gets annoying quickly. Composio also strangely lacks some functions compared to their native counterparts. One of the most prominent examples is how it lacks a 'Pull PRs' function. Compared to native, I have to pull all issues, which includes prs, then filter out the issues. One extra step, but something that seems like it should just exist.